Project Milestone 4 – Progress Report

October 30, 2026

Project title: Machine Learning for Personalized Music Recommendation Based on User Listening Preferences

Team / Author(s): Team 20 ("Twenty"), Course 3052U, Section 46506: David Onyema (project coordinator), Daniel Baba (data manager), Qadeer Khan (model development), Emmanuel Akhigbe (evaluation and testing), Muhammad Mir (documentation and presentation)

Repository: github.com/SMK-BTW/Twenty

Since the baseline memo, we added two new models that use Yambda's precomputed audio embeddings: an Audio-only recommender and a Hybrid that mixes ItemKNN with audio similarity. On the 7-day test window the Hybrid is our best model so far, with NDCG@10 of 0.0315 against 0.0299 for ItemKNN and 0.0151 for MostPopular, and it covers 14.6% of the catalog in users' top 10 lists against 11.7% for ItemKNN.

1. Current Methods and Approaches

1.1 Data

Item	Details
Source(s)	Yandex Yambda-50M from Hugging Face (`yandex/yambda`), pinned to revision `dd6f3a19`…. We use `flat/50m/listens.parquet` for interactions and `embeddings.parquet` for audio vectors. Logged by the Yandex Music streaming service; released for research use only.
Size	46.5M listen events over about 300 days. After cleaning: 9,019,829 user–song pairs from 9,207 users and 630,139 songs. Test catalog: 9,175 users × 67,931 songs with 7,069,539 training pairs (about 1.1% of the user song matrix is filled).
Split strategy	Global time-based split with one cutoff for every user: inner train, then 7 days of validation, then 7 days of test, with a 30-minute gap before each window. Validation is used to tune *k* and the hybrid weight; final numbers come from training on train + validation and scoring the test week.
Class balance	There are no class labels: this is implicit, positive-only feedback, so the "imbalance" is sparsity. In the 100k-row exploration sample, 97.2% of events are listens; likes (1.8%), unlikes (0.7%), dislikes

Item	Details
	(0.2%) and undislikes (<0.1%) are rare. Activity is skewed: users have between 1 and 6,255 training songs.
Known quality issues	`played_ratio_pct` and `track_length_seconds` are missing for 2,781 of 100,000 sampled rows; these are exactly the non-listen events, so listens are complete. Repeat plays create duplicate user–song pairs. Only 72.9% of test-week songs are in our catalog, so the rest can never be hit. Leakage risk from the future is removed by the global cutoff.

Preprocessing and feature engineering. In DuckDB we keep listens with more than 50% of the track played and collapse them to one row per user and song, timestamped at the first such listen. Because of this, every held-out pair is a song that is new to that user. The catalog is the set of songs with at least 20 distinct listeners in the *training window only*, computed after the split so that test listens never decide which songs are eligible. Song popularity (for MostPopular and tie-breaks) is also counted on training data only. For the audio models, each catalog song is matched to its Yambda audio embedding and L2-normalised; 96.4% of catalog songs have one, and the rest get a zero vector and fall back to another score. No feature is fit on validation or test data.

1.2 Models

Stage	Model	Purpose	Key hyperparameters
Baseline	MostPopular	Naive lower bound: the same most-listened songs for every user, minus songs they already heard	None
Intermediate	ItemKNN (cosine)	Collaborative filtering baseline: songs are vectors of their listeners; each song keeps its *k* most similar songs, and a candidate's score is the sum of its similarities to the user's songs	*k* = 20 (searched 20, 50, 100, 200)
Main (ablation)	Audio-only	Content-based model: a user profile is the mean of the audio vectors of their songs; candidates are ranked by cosine similarity to that profile	Falls back to popularity when audio is missing
Main	Hybrid (ItemKNN + audio)	Primary candidate: score = $\alpha \cdot$ KNN + $(1 - \alpha) \cdot$ audio, both scaled to 0–1 per user	α = 0.5 (searched 1.0, 0.75, 0.5, 0.25, 0.0); *k* = 20

1.3 Training and Evaluation Setup

- **Framework and tooling:** Python, DuckDB 1.1.3, NumPy 2.1.3, pandas 2.2.3, SciPy 1.14.1 (sparse matrices), scikit-learn 1.6.1 (`NearestNeighbors`, brute-force cosine), matplotlib for plots. Code lives in one Jupyter notebook under Git on GitHub.
- **Hardware:** CPU only, a Windows laptop with 8 GB RAM. Tuning four *k* values on validation took about 8 minutes; scoring both baselines on test took about 2 minutes. The interaction data is about 750 MB; the audio embedding file is about 13.8 GB.
- **Loss function and optimiser:** None. ItemKNN, Audio-only and the Hybrid are memory-based models with no learned weights, so there is no loss or gradient training; "fitting" means building the item–item similarity matrix and the audio profiles.
- **Regularisation:** Complexity is controlled by the neighbourhood size *k* (small *k* keeps only strong similarities) and by the 20-listener catalog threshold, which removes songs with unreliable similarity estimates.
- **Hyperparameter search:** Grid search on the validation week only: $k \in \{20, 50, 100, 200\}$ for ItemKNN, then $\alpha \in \{1.0, 0.75, 0.5, 0.25, 0.0\}$ for the Hybrid. Selection metric: NDCG@10. Test data is never used to choose settings.
- **Evaluation metrics:** NDCG@10 is primary because it rewards putting the right songs near the top of a short list, which is what a user sees. Secondary metrics are Precision, Recall and HitRate at 10 and 50, plus catalog coverage (share of catalog songs recommended to anyone) to measure diversity. Every check also asserts that no user is recommended a song they already heard.
- **Reproducibility:** Pinned library versions in the first notebook cell, a pinned dataset revision, and a deterministic pipeline (no random steps; the exploration sample uses `REPEATABLE (42)`; ties are broken by popularity, then song id). Repository: github.com/SMK-BTW/Twenty.

2. Theoretical Justification for Chosen Methods

2.1 Choice of Model Family

Claim: item-based nearest neighbours are a strong, appropriate first model for implicit music feedback. **Reasoning:** listeners cluster around artists, genres and moods, so two songs heard by many of the same people are likely to suit the same new listener. Item–item similarities are also more stable than user–user ones when there are far more songs than users, and the scores are easy to explain ("recommended because you played X"). **Evidence:** Sarwar et al. (2001) and Deshpande and Karypis (2004) introduced item-based top-N recommendation for exactly this setting, and Ferrari Dacrema et al. (2019) showed that well-tuned neighbourhood methods often match or beat newer neural recommenders.

Claim: adding audio similarity should help where listening data is thin. **Reasoning:** collaborative filtering can only relate songs that share listeners, so it favours popular songs and cannot place rarely played ones; audio content describes every song regardless of play counts. **Evidence:** van den Oord et al. (2013) showed that audio-based representations can recommend music with little usage data, and Burke (2002) describes weighted hybrids, which combine scores from several recommenders, as a simple way to cover each one's blind spots. Our Hybrid is a weighted hybrid of this kind.

MostPopular is kept as the naive benchmark because popularity is a surprisingly hard baseline to beat on top-N tasks (Cremonesi et al., 2010).

2.2 Choice of Loss Function and Optimisation

None of our current models minimises a loss: they are memory-based, so their "training" is computing cosine similarities and profiles directly from the data. Cosine similarity suits binary listen vectors because it normalises for how popular each song is, so two songs are similar when they share a large *fraction* of their listeners, not just many listeners. For the Hybrid, both scores are rescaled to 0–1 per user before mixing; without this, whichever score has the larger raw range would dominate regardless of α. A learned model with a ranking loss, such as BPR (Rendle et al., 2009) or implicit ALS (Hu et al., 2008), is the planned next step (Section 6).

2.3 Bias-Variance and Generalisation

The neighbourhood size *k* is the main capacity control. A small *k* keeps only each song's closest neighbours, which gives sharper, more varied recommendations; a large *k* adds weak, noisy neighbours that mostly point to popular songs. Validation supports the smallest value tried: NDCG@10 was 0.0310 at $k = 20$ and between 0.0302 and 0.0310 for larger *k*, while catalog coverage@10 fell from 12.5% to 9.4% as *k* grew. ItemKNN also generalised well across time: its NDCG@10 moved only from 0.0310 (validation) to 0.0299 (test), while MostPopular dropped from 0.0191 to 0.0151, because the most popular songs change from week to week. The hybrid weight behaved similarly, with $\alpha = 0.5$ best on validation (0.0333) and the test result (0.0315) still above ItemKNN.

2.4 Evaluation Methodology

A global time-based split is used because a recommender is always asked to predict the future from the past; a random split would let the model learn from listens that happen after the ones it is tested on (look-ahead bias). The same cutoff for every user also prevents one user's future from leaking into another user's training data through shared songs. The 30-minute gaps stop a single listening session from being split across train and test, following the Yambda paper's protocol (Ploshkin et al., 2025). Testing only on first-time listens makes the task "find new music", which is the goal of the project. NDCG@k (Järvelin and Kekäläinen, 2002) is the primary metric because order matters in a short list, and coverage is added because accuracy alone rewards recommending the same hits to everyone.

2.5 Alternatives Considered

Alternative	Why it was not chosen (yet)
User-based KNN	About 9,000 users with very different activity levels (1 to 6,255 songs) make user–user similarities noisy; item-based is the standard choice for this shape of data
Matrix factorisation (iALS, BPR)	A good learned model for this data, but it adds training time and hyperparameters; deferred to after the audio study (Section 4)

Alternative	Why it was not chosen (yet)
Sequential / transformer models (e.g. SASRec)	Need GPU training and much more tuning than our 8 GB CPU laptops allow within the course timeline
Training our own audio model on raw audio	Yambda does not release playable audio, only precomputed embeddings, so we use those directly
Random train/test split	Would leak future listening into training and overstate results

3. Preliminary Results and Findings

3.1 Quantitative Results

The Hybrid is the best model on the primary metric, and ItemKNN roughly doubles MostPopular on every accuracy metric. Our models have no training loss, so a "train" score is not meaningful; validation is the week used for tuning and test is the final held-out week.

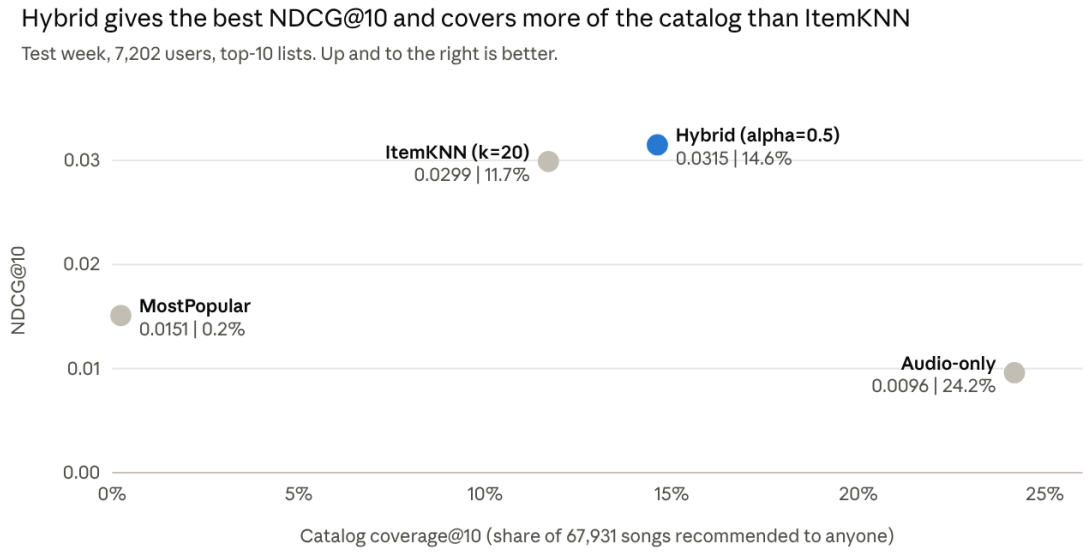
Model	Validation NDCG@10	Test NDCG@10	Test Recall@10	Test HitRate@10	Test Coverage@10	Fit + scoring time
MostPopular	0.0191	0.0151	0.0067	8.8%	0.2%	Under 1 min
ItemKNN (k = 20)	0.0310	0.0299	0.0116	17.3%	11.7%	About 2 min per run
Audio-only	≈0.0104	0.0096	0.0033	7.7%	24.2%	Not yet timed
Hybrid (α = 0.5)	0.0333	**0.0315**	**0.0122**	**17.9%**	14.6%	Not yet timed

At a list length of 50 the picture changes: the Hybrid and ItemKNN are almost tied on accuracy, but the Hybrid reaches far more of the catalog.

Model	Test NDCG@50	Test Recall@50	Test HitRate@50	Test Coverage@50
MostPopular	0.0158	0.0179	20.3%	0.5%
ItemKNN (k = 20)	0.0353	**0.0427**	**42.2%**	32.7%
Audio-only	0.0113	0.0123	22.9%	**56.7%**
Hybrid (α = 0.5)	**0.0362**	0.0419	41.9%	45.6%

The Audio-only validation figure is the α = 0 row of the hybrid search, which is audio scoring with a KNN fallback for songs without audio. All results come from one deterministic run, so there is no spread across seeds yet; confidence intervals are planned (Section 6).

3.2 Visualisations



Machine_Learning_Project.ipynb, 'Comparing all 4 models' cell · test week, 7,202 users

The chart shows the main trade-off: Audio-only reaches the most songs but ranks poorly, while the Hybrid keeps ItemKNN's accuracy and adds coverage. Learning curves, confusion matrices and ROC curves from the template do not apply, because there is no iterative training and no classification. Other figures available in the notebook: the bar chart of NDCG@10, Recall@10 and Coverage@10 for all four models, the *k*-tuning table, and the α-tuning table.

3.3 Key Findings

1. **Audio helps when combined with listening history.** The Hybrid raised NDCG@10 from 0.0299 to 0.0315 (+5.4% relative) and Recall@10 from 0.0116 to 0.0122 over ItemKNN, and lifted coverage@10 from 11.7% to 14.6%. The validation search agreed: an even 50/50 mix beat both pure KNN (0.0310) and mostly-audio mixes.
2. **The gain is at the top of the list, not in how many songs are found.** At 50 recommendations, Recall@50 is 0.0419 for the Hybrid versus 0.0427 for ItemKNN. Audio mainly reorders and diversifies the list rather than retrieving more correct songs.
3. **Audio alone is not enough.** Audio-only scored NDCG@10 of 0.0096, below even MostPopular (0.0151), despite the highest coverage (24.2% at 10, 56.7% at 50). Sounding similar to a user's history does not mean they will play it in the next week.
4. **ItemKNN is a solid, stable baseline.** It doubles MostPopular on NDCG@10 (0.0299 vs 0.0151) and HitRate@10 (17.3% vs 8.8%), and dropped only 0.0011 from validation to test, while MostPopular dropped 0.0040.

5. **We are still well below the Yambda paper's ItemKNN.** Our 0.0299 is about 38% of the paper's reported 0.0781, while our MostPopular (0.0151) is close to theirs (0.0186). Section 3.4 and our feedback response (Section 7) discuss why.

3.4 Error Analysis

- **Heavy listeners are served worst.** Splitting test users into thirds by training history, ItemKNN's NDCG@10 is 0.0421 for light users (1–401 songs), 0.0272 for medium (402–992) and 0.0203 for heavy (993–6,255). Our hypothesis is that summing similarities over hundreds of songs pulls every heavy user toward the same well-connected "hub" songs, while the new songs they actually find are more niche.
- **Strong popularity bias remains.** 79.5% of ItemKNN's top-10 picks come from the 10% most-listened songs, and the average pick sits at the 91.5th popularity percentile. ItemKNN recommends 7,941 distinct songs across users against MostPopular's 154, but most are still well-known songs. The Hybrid's higher coverage suggests audio partly counters this; we have not yet run the same diagnostic on it.
- **Recommendation-driven listens are harder for popularity.** MostPopular's recall@10 on organic (user-chosen) listens is 0.0053 but only 0.0027 on listens started by Yandex's own recommender; ItemKNN is more even (0.0076 vs 0.0069).
- **Some misses cannot be fixed by the model.** 27.1% of test-week songs fall outside our 20-listener catalog, so every one of them is a guaranteed miss.

3.5 Limitations of Current Results

- One deterministic run on a single 7-day test week, with no confidence intervals or significance test. The Hybrid's +0.0016 NDCG@10 gain may not be statistically meaningful yet.
- Our protocol (first-time listens only, 7-day window, 20-listener catalog) differs from the Yambda paper's, so the two sets of numbers are not directly comparable.
- The Audio-only and Hybrid models silently fall back to popularity or KNN for the 3.6% of songs without audio, which slightly mixes the comparison.
- The user-activity and popularity diagnostics have only been run for MostPopular and ItemKNN, not for the audio models.
- Hyperparameter grids are small (4 values of k, 5 values of α), and k was not re-tuned for the Hybrid.
- The saved notebook was not run top to bottom in one session (the audio-loading cell was interrupted and rerun out of order), so a clean end-to-end rerun is still needed.

4. Deviations from the Original Plan

The main change is that, following TA feedback, we added audio-based models in place of the matrix-factorisation models our baseline memo scheduled for October 20–30. The data split, cleaning and evaluation protocol are unchanged, and Milestone 4 was submitted on its October 30 deadline.

Planned	What actually happened	Reason	Impact
Build iALS and BPR models (Oct 20–30, baseline memo)	Built Audio-only and a Hybrid (ItemKNN + audio) instead; iALS and BPR not started	The TA suggested using Yambda's audio embeddings; our diagnostics also showed strong popularity bias that content features can address	The Hybrid became our main model; iALS/BPR are now an optional stretch goal for Milestone 5
Use only the interaction files (about 750 MB)	Added the 13.8 GB audio embedding file	Needed for the audio models	Much heavier memory and download needs (Section 5)
Add 95% confidence intervals to all metrics	Not done yet	Time went into the audio models	Cannot yet say whether the Hybrid's gain is significant; moved to Milestone 5
Run a second evaluation that allows repeat listens	Not done yet	Same as above	The gap with the Yambda paper is still unexplained; moved to Milestone 5
Try a popularity model weighted toward recent listens	Not done yet	Lower priority than the audio study	Dropped unless time allows
Separate scripts, a `tests/` folder and a tagged code release	Everything stays in one notebook; no tests folder or release yet	One notebook is easier for all five members to run, including on Colab	Code clean-up and release moved to Milestone 6
Exploration on `multi_event.parquet`	Main pipeline uses `listens.parquet` (the listen-only subset)	Our task only uses listens; the smaller file is faster	None; results are equivalent for listens

5. Current Blockers and Challenges

Our biggest challenge is compute: the audio embeddings are 13.8 GB and we work on 8 GB laptops, which makes every audio experiment slow and fragile.

Challenge	Severity	Effect on project	Mitigation / support needed	Owner
Audio embeddings (13.8 GB) are too	High	Loading was interrupted at	Load only the 67k catalog songs with a 2 GB DuckDB memory	Daniel Baba

Challenge	Severity	Effect on project	Mitigation / support needed	Owner
large to load comfortably on 8 GB RAM		least once; audio runs are slow and hard to repeat	limit (done); cache the matched vectors to a small local file; use Colab or a lab machine for full reruns	
ItemKNN is far below the Yambda paper (0.0299 vs 0.0781 NDCG@10)	Medium	Makes it hard to tell whether our baseline is weak or just evaluated differently	Rerun with the paper's protocol (repeat listens allowed, 1-day test window, full catalog) to isolate the cause	Emmanuel Akhigbe
No confidence intervals on a small Hybrid gain (+0.0016 NDCG@10)	Medium	We cannot yet claim the improvement is real	Bootstrap over test users (e.g. 1,000 resamples) for 95% intervals and a paired comparison	Emmanuel Akhigbe
Heavy listeners get much worse recommendations (NDCG@10 0.0203 vs 0.0421 for light users)	Medium	Limits overall accuracy	Down-weight very popular neighbours or normalise each user's history; check whether audio helps this group	Qadeer Khan
Full pipeline takes 10–15 minutes per run on a laptop	Low	Slows tuning	Develop on a 20% user sample and run the full data only for final numbers	David Onyema

Help requested from supervisor or peers: access to a machine with at least 16 GB RAM (or guidance on course compute) for the audio experiments, and advice on whether matching the Yambda paper's protocol exactly is expected, or whether explaining the difference is enough.

6. Revised Timeline and Next Steps

Data, baselines and the first improved model are done; the remaining work is significance testing, error analysis and robustness checks for Milestone 5 (Nov 18), with the final deliverable due Dec 4.

6.1 Revised Timeline

Phase	Tasks	Original date	Revised date	Status
Data collection and cleaning	Choose and verify Yambda-50M, define the ML problem, split and metric (Milestone 2)	Sept 30, 2026	Sept 30, 2026	Complete

Phase	Tasks	Original date	Revised date	Status
Baseline and initial models	Data pipeline, MostPopular, ItemKNN with *k* tuning, diagnostics, baseline memo (Milestone 3)	Oct 14, 2026	Oct 19, 2026	Complete
Model improvement and tuning	Audio-only and Hybrid models, α tuning, comparison with baselines (Milestone 4)	Oct 28, 2026	Oct 30, 2026	Complete
Ablations and robustness testing	95% bootstrap intervals and paired tests; error analysis by user, song and song source; second time window; audio-fallback and catalog-threshold ablations; repeat-listen rerun vs the Yambda paper (Milestone 5)	Nov 18, 2026	Nov 18, 2026	Not started
Final evaluation on held-out test set	One clean top-to-bottom notebook run; final comparison table and figures (Milestone 5–6)	Nov 18, 2026	Nov 24, 2026	Not started
Final report and presentation	Clean code, tagged release, final report, slides and demo (Milestone 6)	Dec 2, 2026	Dec 2, 2026 (final deliverable due Dec 4)	Not started

6.2 Prioritised Next Steps

1. **Uncertainty:** bootstrap 95% confidence intervals over test users for every metric, plus paired tests of each model against ItemKNN. Owner: Emmanuel Akhigbe. Due Nov 10 (team meeting). Expected outcome: we know whether the Hybrid's +0.0016 NDCG@10 gain is real.
2. **Error analysis:** break down misses by user activity, song popularity and song source (organic vs Yandex-recommended), and run the popularity-bias diagnostics on the Hybrid. This also tests whether our models partly copy Yandex's own recommender. Owners: Emmanuel Akhigbe and Qadeer Khan. Due Nov 14. Expected outcome: a clear list of where each model fails.
3. **Robustness:** repeat the comparison on a second time window, ablate the audio fallback and the 20-listener catalog threshold, and rerun under the Yambda paper's repeat-listen protocol. Owners: Daniel Baba and Qadeer Khan. Due Nov 18. Expected outcome: evidence that the results hold beyond one split, and a measured reason for the gap with the paper.

6.3 Risks to the Revised Plan and Contingencies

- **If we have time for iALS/BPR but training does not fit in laptop memory:** train on the 20% user sample, or submit the Hybrid as our final model and document the learned models as future work.
- **If the Hybrid's gain is not significant:** report it honestly as a coverage improvement at equal accuracy, which is still a useful result for music discovery.
- **If the audio file cannot be reloaded reliably:** cache the 67k matched catalog vectors once to a small local file and rerun from that file.

7. Responses to Feedback Received So Far

We acted on the TA's suggestion to use audio embeddings and accepted all three suggestions from the peer check-in; the audio work is done and the rest is planned for Milestone 5.

Feedback (source, date)	Our response	Action taken	Status
Add Yambda's audio embeddings to the listening-history baseline (TA, Milestone 3 feedback)	Agreed. Audio describes every song regardless of play counts, so it could help where listening data is thin	Built Audio-only and Hybrid models; the Hybrid beats ItemKNN on NDCG@10 (0.0315 vs 0.0299)	Done
Look in depth at why our ItemKNN scores lower than the Yambda paper's, since song similarity alone should not explain the gap (peer group, Oct 24, 2026)	Agreed. Our MostPopular is close to the paper's, so the gap is specific to ItemKNN and most likely comes from protocol differences: we test only first-time listens, use a 7-day instead of a 1-day test window, drop songs with fewer than 20 listeners, and tried fewer *k* values	Listed and documented the likely causes; added diagnostics by user activity and popularity; a rerun under the paper's protocol is scheduled	In progress (due Nov 18)
Look for ways to improve the model's performance (peer group, Oct 24, 2026)	Agreed. Our diagnostics showed strong popularity bias, so we added content information that does not depend on play counts	The Hybrid raised NDCG@10 from 0.0299 to 0.0315 and coverage@10 from 11.7% to 14.6%	Done
Test other models to see if any beat the current one (peer group, Oct 24, 2026)	Agreed	Added Audio-only and Hybrid models; iALS and BPR are a stretch goal for Milestone 5	Partly done

Feedback (source, date)	Our response	Action taken	Status
Keep the comparison with the published paper and the clear results tables (peer group, Oct 24, 2026, noted as strengths)	We will keep both in the final report	Paper comparison and results tables carried into this report	Ongoing

9. Midpoint Peer Check-In

Peer / group consulted: Group 26, "Related Romance Languages", working on language identification with kNN

Date and format: October 24, 2026; live discussion over Discord where each team presented its progress and gave feedback

9.1 What We Shared

We walked the other team through our baseline stage: the Yambda-50M data and cleaning steps, the time-based train/validation/test split, the MostPopular and ItemKNN models, and our test results (NDCG@10 of 0.0151 and 0.0299). We also showed how our numbers compare with the Yambda paper and our possible reasons for the differences.

9.2 Feedback Received from Peers

- **Strengths noted:** The other team found the goal relevant, since recommendation systems drive large streaming services such as Spotify. They saw the size of our data (millions of user–song interactions) as a strength. They said our results were laid out clearly, making it easy to see how each model did, and they valued that we benchmarked against a published paper and tried to explain where our numbers differ from it.
- **Concerns or questions raised:** Their main question was why our ItemKNN falls so far below the paper's result. Because the model is built on song-to-song similarity, much as in the paper, they suspected something else in our setup was holding it back.
- **Suggestions:** Investigate the cause of that gap in more detail, look into concrete ways to raise the model's performance, and try other model types to see whether any of them beat our current one.

9.3 Feedback We Gave to Peers

- **Peer project:** Related Romance Languages (Group 26), a kNN-based model that identifies closely related Romance languages
- **Strengths:** Their kNN model was very accurate, at about 96.7%. We felt their dataset fit their research question well and that their data preparation was careful. Their comparison of model performance was clear and easy to follow.
- **Questions and gaps:** We asked whether the accuracy would hold up on short pieces of text, which carry fewer clues about the language. We also pointed out that the model confused

closely related languages, such as Catalan and Occitan, and asked whether a different algorithm might reach higher accuracy.
- **Constructive suggestions:** Evaluate the model on shorter and harder text samples to see whether it still identifies languages correctly, and benchmark the kNN model against other machine-learning approaches to find out which performs best.

9.4 Changes Resulting from the Check-In

- **New models:** We built the Audio-only and Hybrid models, and the Hybrid now beats ItemKNN on NDCG@10 (0.0315 vs 0.0299). iALS and BPR remain a stretch goal.
- **Performance work:** We targeted the popularity bias found in our diagnostics by adding audio similarity, which raised catalog coverage@10 from 11.7% to 14.6%.
- **Paper gap:** We added a planned rerun under the Yambda paper's evaluation protocol (Section 6.2) so the difference can be measured instead of guessed.
- **Timeline:** These changes moved the confidence intervals and the paper-protocol rerun into Milestone 5, due Nov 18 (Section 6).

12. Conclusion

We have a working, leakage-free pipeline on Yambda-50M, two baselines, and a Hybrid model that beats ItemKNN on NDCG@10 (0.0315 vs 0.0299) while recommending a wider range of songs. Our top risk is compute: the 13.8 GB audio data strains our 8 GB laptops. With the fallback of submitting the Hybrid as our final model, we are confident of delivering the final outcome, and the remaining work is about confirming the gain with confidence intervals and explaining the gap with the published results.

References

1. Burke, R. 2002. Hybrid Recommender Systems: Survey and Experiments. *User Modeling and User-Adapted Interaction*, 12(4).
2. Cremonesi, P., Koren, Y. and Turrin, R. 2010. Performance of Recommender Algorithms on Top-N Recommendation Tasks. *Proceedings of ACM RecSys 2010*.
3. Deshpande, M. and Karypis, G. 2004. Item-Based Top-N Recommendation Algorithms. *ACM Transactions on Information Systems*, 22(1).
4. Ferrari Dacrema, M., Cremonesi, P. and Jannach, D. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. *Proceedings of ACM RecSys 2019*.
5. Hu, Y., Koren, Y. and Volinsky, C. 2008. Collaborative Filtering for Implicit Feedback Datasets. *Proceedings of IEEE ICDM 2008*.
6. Järvelin, K. and Kekäläinen, J. 2002. Cumulated Gain-Based Evaluation of IR Techniques. *ACM Transactions on Information Systems*, 20(4).
7. Ploshkin, A., Baikalov, V., Burlakov, D., Savushkin, N., et al. 2025. Yambda-5B: A Large-Scale Multi-modal Dataset for Ranking and Retrieval. arXiv:2505.22238. arxiv.org/abs/2505.22238
8. Rendle, S., Freudenthaler, C., Gantner, Z. and Schmidt-Thieme, L. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. *Proceedings of UAI 2009*.
9. Sarwar, B., Karypis, G., Konstan, J. and Riedl, J. 2001. Item-Based Collaborative Filtering Recommendation Algorithms. *Proceedings of WWW 2001*.

10. van den Oord, A., Dieleman, S. and Schrauwen, B. 2013. Deep Content-Based Music Recommendation. *Advances in Neural Information Processing Systems 26 (NeurIPS 2013)*.
11. Yandex. 2025. Yambda dataset, Hugging Face (`yandex/yambda`). huggingface.co/datasets/yandex/yambda